

INVENTORY MANAGEMENT

AGENTIC SYSTEM

Complete Database Design Specification

Version: 1.0
Total Databases: 4
Total Tables: 18 | Views: 1
Total Agents: 8
SKUs: 5,000 | Locations: 53 | Inventory Records: 265,000
Suppliers: 35

1. System Overview

This specification covers the complete relational database design powering an Inventory Management Agentic System. The system consists of eight autonomous agents, each responsible for a distinct operational domain — from real-time stock monitoring and demand forecasting to supplier evaluation and automated procurement. All agents read from and write to a shared set of four logical databases containing 18 tables and one derived view.

1.1 Business Scale

Dimension	Count / Range	Notes
Retail Stores	50	Spread across 10 US states
Distribution Centers	3	Western, Central, East Coast
Store Formats	3	Small 5k / Medium 12k / Large 25k sq ft
Total Locations	53	Stores + DCs combined
Product SKUs	5,000	8 product categories
Inventory Records	265,000	53 locations × 5,000 SKUs
Active Suppliers	35	Lead time 2–21 days
In-Transit Inventory	~8% of total	Average at any given time

1.2 Inventory Status Distribution

Status	% of Records	Record Count	Action Triggered
BELOW_SAFETY	15%	~39,750	Replenishment / Transfer alert
OPTIMAL	60%	~159,000	No action needed
OVERSTOCK	20%	~53,000	Transfer Optimization Agent
ZERO_STOCK	5%	~13,250	Emergency replenishment PO

2. Database 1 — Stores & Warehouses

This database stores the physical geography of the retail network: states, store formats, distribution centers, individual stores, and the master location table that unifies stores and DCs under a single identifier for use in inventory and transit tables.

2.1 Business Requirements

- 50 retail stores across 10 US states (CA, TX, FL, NY, IL, OH, GA, NC, MI, PA)
- 3 regional distribution centers: Western Coast DC, Central DC, East Coast DC
- 3 store formats: Small (5,000 sq ft), Medium (12,000 sq ft), Large (25,000 sq ft)
- Each store is assigned to exactly one DC and one state
- The locations master table allows inventory tables to reference stores and DCs interchangeably

2.2 Table Schemas

states

Lookup table for the 10 US states in which stores operate.

Column Name	Data Type	Key	Description / Constraints
state_id	INT	PK	Surrogate primary key
state_name	VARCHAR(50)		Full state name e.g. California
state_code	VARCHAR(5)		2-letter code e.g. CA

store_formats

Defines the three physical store size categories.

Column Name	Data Type	Key	Description / Constraints
format_id	INT	PK	Surrogate primary key
format_name	VARCHAR(20)		Small / Medium / Large
square_feet	INT		Floor area: 5000, 12000, or 25000 sq ft

distribution_centers

The three regional DCs that supply replenishment stock to stores.

Column Name	Data Type	Key	Description / Constraints
dc_id	INT	PK	Surrogate primary key
dc_name	VARCHAR(100)		e.g. Western Coast DC
region	VARCHAR(50)		Western / Central / East Coast
city	VARCHAR(50)		City where DC is located

Column Name	Data Type	Key	Description / Constraints
state_id	INT	FK	References states(state_id)

stores

Core store master table. Every inventory record traces back here via location_id.

Column Name	Data Type	Key	Description / Constraints
store_id	INT	PK	Surrogate primary key
store_code	VARCHAR(20)	UQ	Unique human-readable code e.g. STR-001
store_name	VARCHAR(100)		Display name of the store
city	VARCHAR(100)		City where store is located
state_id	INT	FK	References states(state_id)
format_id	INT	FK	References store_formats(format_id)
dc_id	INT	FK	Assigned distribution center
opening_date	DATE		Store opening date
active_flag	BOOLEAN		TRUE = operational, FALSE = closed/inactive

locations (master)

Unified lookup for all 53 locations (50 stores + 3 DCs). Inventory and transit tables use location_id from this table so agent queries need only one join regardless of whether they are looking at a store or a DC.

Column Name	Data Type	Key	Description / Constraints
location_id	INT	PK	Surrogate primary key
location_name	VARCHAR(100)		Display name mirroring store or DC name
location_type	VARCHAR(20)		STORE or DC
state_id	INT	FK	References states(state_id)
active_flag	BOOLEAN		Whether location is currently active

3. Database 2 — Product Catalog

This database holds all 5,000 SKUs along with their classification attributes: category, velocity class, and optional seasonality. These attributes drive the logic of the Demand Forecast Agent and directly influence reorder quantities calculated by the Replenishment Planning Agent.

3.1 Business Requirements

- 5,000 active SKUs across 8 categories: Electronics, Apparel, Home Goods, Sporting Goods, Toys, Health & Beauty, Grocery, Seasonal
- Retail price range: \$4.99 to \$1,299.99 — captured as unit_cost and retail_price
- Velocity distribution: 20% Fast-moving (A), 30% Medium (B), 50% Slow (C)
- Three seasonal peaks: Holiday (Nov–Dec), Back-to-School (Aug–Sep), Summer (Jun–Jul)
- Non-seasonal SKUs have season_id = NULL — this is intentional and expected
- Additional flags: hazmat, temperature-controlled, shelf-life — critical for logistics routing

3.2 Table Schemas

product_categories

The 8 top-level product categories. Used by both product and supplier tables to create category-supplier mappings.

Column Name	Data Type	Key	Description / Constraints
category_id	INT	PK	Surrogate primary key
category_name	VARCHAR(100)		e.g. Electronics, Apparel, Grocery
category_description	VARCHAR(500)		Describes what the category covers

velocity_classes

Classifies SKUs by how fast they sell. Used by Demand Forecast Agent to weight forecast models.

Column Name	Data Type	Key	Description / Constraints
velocity_id	INT	PK	Surrogate primary key
velocity_code	CHAR(1)		A = Fast, B = Medium, C = Slow
velocity_name	VARCHAR(50)		Descriptive label e.g. Fast Moving
percentage_distribution	DECIMAL(5,2)		Share of catalog: 20%, 30%, 50%

seasonal_patterns

Defines seasonal demand windows and their multipliers. A demand_multiplier of 2.5 means 2.5x baseline demand during that window.

Column Name	Data Type	Key	Description / Constraints
season_id	INT	PK	Surrogate primary key

Column Name	Data Type	Key	Description / Constraints
season_name	VARCHAR(100)		Holiday Peak / Back To School / Summer
start_month	INT		1–12 representing month number
end_month	INT		1–12 representing month number
demand_multiplier	DECIMAL(5,2)		Multiplier applied to base demand forecast

products

Core SKU master table. The single most referenced table across the entire system — every inventory, transit, and procurement record traces back here.

Column Name	Data Type	Key	Description / Constraints
sku_id	INT	PK	Surrogate primary key
sku_code	VARCHAR(50)	UQ	Human-readable unique SKU code
product_name	VARCHAR(255)		Full product display name
category_id	INT	FK	References product_categories(category_id)
velocity_id	INT	FK	References velocity_classes(velocity_id)
season_id	INT NULL	FK	Optional — NULL for year-round products
brand_name	VARCHAR(100)		Brand of the product
unit_cost	DECIMAL(10,2)		Wholesale cost — used in PO calculations
retail_price	DECIMAL(10,2)		Selling price range \$4.99–\$1,299.99
weight_lbs	DECIMAL(8,2)		For freight cost and DC capacity planning
shelf_life_days	INT NULL		NULL for non-perishables; days for Grocery/Health
active_flag	BOOLEAN		FALSE = discontinued SKU
launch_date	DATE		When this SKU became active
reorderable_flag	BOOLEAN		Can replenishment orders be placed?
hazmat_flag	BOOLEAN		Triggers special logistics routing
temperature_controlled_flag	BOOLEAN		Requires cold-chain shipment
min_order_qty	INT		Minimum units per PO line
case_pack_size	INT		Units per case for warehouse counting

4. Database 3 — Inventory Position

This is the operational heart of the system. With 265,000 records (53 locations x 5,000 SKUs), it represents the real-time state of all stock. The Inventory Monitoring Agent reads this database every cycle to identify anomalies. The `in_transit_inventory` table provides visibility into inbound stock to prevent premature reorder decisions.

4.1 Business Requirements

- Track current, reserved, and available stock for every SKU at every location
- Maintain safety stock thresholds, reorder points, and max stock levels per SKU-location
- Status classification: ZERO_STOCK (5%), BELOW_SAFETY (15%), OPTIMAL (60%), OVERSTOCK (20%)
- Approximately 8% of total inventory is in transit at any given time
- Transit statuses: IN_TRANSIT, DELAYED, ARRIVED, LOST
- The `inventory_health` view provides a single computed status per record without duplicating logic

4.2 Table Schemas

inventory_positions

Core inventory record — one row per SKU-location combination (265,000 rows total). The `inventory_status` field is stored but also validated by the `inventory_health` view.

Column Name	Data Type	Key	Description / Constraints
inventory_id	BIGINT	PK	BIGINT for large-table scale
sku_id	INT	FK	References products(sku_id)
location_id	INT		Soft reference to locations(location_id) — no FK declared for performance
location_type	VARCHAR(20)		STORE or DC — redundant but avoids joins in hot queries
current_stock	INT		Physical units currently on hand
reserved_stock	INT		Units allocated to pending orders
available_stock	INT		current_stock minus reserved_stock
safety_stock	INT		Minimum buffer before reorder alert fires
reorder_point	INT		Trigger level for Replenishment Agent
max_stock_level	INT		Upper bound — exceeding this triggers overstock alert
inventory_status	VARCHAR(20)		ZERO_STOCK / BELOW_SAFETY / OPTIMAL / OVERSTOCK
last_updated	TIMESTAMP		When this record was last modified

in_transit_inventory

Tracks all inbound shipments currently moving between a source and destination location. Used by Replenishment and Transfer agents to avoid double-ordering.

Column Name	Data Type	Key	Description / Constraints
transit_id	BIGINT	PK	BIGINT for high-volume shipment records
sku_id	INT	FK	References products(sku_id)
source_location_id	INT		Originating store or DC (soft FK to locations)
destination_location_id	INT		Receiving store or DC (soft FK to locations)
quantity	INT		Units in this shipment
shipment_date	DATE		Date shipment left source location
expected_arrival_date	DATE		Promised delivery date
transit_status	VARCHAR(30)		IN_TRANSIT / DELAYED / ARRIVED / LOST
carrier_name	VARCHAR(100)		Logistics carrier handling the shipment

inventory_health (Derived View)

This view wraps the CASE logic that classifies each inventory record's status. By centralising the classification here, all agents reference a single authoritative definition — preventing agents from implementing slightly different threshold logic independently.

Condition	Assigned Status
current_stock = 0	ZERO_STOCK — immediate emergency replenishment needed
current_stock < safety_stock	BELOW_SAFETY — replenishment alert triggered
current_stock > max_stock_level	OVERSTOCK — transfer optimization or markdown needed
All other cases	OPTIMAL — no action required

5. Database 4 — Supplier Management

This database governs all 35 vendor relationships. It is the exclusive data source for the Supplier Evaluation, Policy Approval, and Procurement agents. It tracks not just contact and contract information, but also performance history, risk scores, and volume-based pricing tiers — giving the Procurement Agent the information needed to select the optimal supplier and price point for every PO.

5.1 Business Requirements

- 35 active vendors with lead times ranging from 2 to 21 days
- Volume discount pricing at three thresholds: 100 units, 500 units, 1,000 units
- Performance tracked per evaluation period: on-time delivery 85–99%, quality score 90–100, fill rate 92–100%
- Payment terms include NET30, NET60, and 2/10 NET30 (2% discount if paid within 10 days)
- Risk profiles score financial, operational, and geographic risk independently
- Supplier-to-category mapping defines which suppliers can fulfill which product categories and their respective lead times

5.2 Table Schemas

payment_terms

Lookup table for the three payment term structures. Referenced by suppliers.

Column Name	Data Type	Key	Description / Constraints
payment_term_id	INT	PK	Surrogate primary key
payment_term_code	VARCHAR(50)		NET30 / NET60 / 2/10 NET30
description	VARCHAR(200)		Plain-language description of terms

suppliers

Master vendor table. The hub of DB4 — all other supplier tables join here.

Column Name	Data Type	Key	Description / Constraints
supplier_id	INT	PK	Surrogate primary key
supplier_code	VARCHAR(30)	UQ	Unique vendor reference code
supplier_name	VARCHAR(200)		Full legal vendor name
supplier_region	VARCHAR(100)		Geographic region of supplier
contact_email	VARCHAR(200)		Primary contact email
contact_phone	VARCHAR(50)		Primary contact phone number
active_flag	BOOLEAN		FALSE = suspended or expired contract
preferred_supplier	BOOLEAN		TRUE = Procurement Agent prioritises this vendor
contract_start_date	DATE		When current contract became effective

Column Name	Data Type	Key	Description / Constraints
contract_end_date	DATE		Policy Approval Agent checks this before approving POs
payment_term_id	INT	FK	References payment_terms(payment_term_id)

supplier_category_mapping

Many-to-many bridge between suppliers and product categories. Also stores the lead time specific to that supplier-category pair.

Column Name	Data Type	Key	Description / Constraints
supplier_category_id	INT	PK	Surrogate primary key
supplier_id	INT	FK	References suppliers(supplier_id)
category_id	INT	FK	References product_categories(category_id)
lead_time_days	INT		Days from PO placement to DC arrival for this category

supplier_pricing_tiers

Volume discount schedule per supplier-category pair. Three tiers at 100, 500, and 1,000 minimum units.

Column Name	Data Type	Key	Description / Constraints
pricing_tier_id	INT	PK	Surrogate primary key
supplier_id	INT	FK	References suppliers(supplier_id)
category_id	INT	FK	References product_categories(category_id)
minimum_quantity	INT		100, 500, or 1000 units
discount_percentage	DECIMAL(5,2)		Percentage discount applied above that threshold

supplier_performance_metrics

Historical performance snapshots per vendor per evaluation period. Multiple rows per supplier over time.

Column Name	Data Type	Key	Description / Constraints
metric_id	BIGINT	PK	BIGINT for growing historical log
supplier_id	INT	FK	References suppliers(supplier_id)
evaluation_date	DATE		Date of performance measurement
on_time_delivery_rate	DECIMAL(5,2)		85–99% range; below 85% flags risk
quality_score	DECIMAL(5,2)		90–100 scale based on defect rates
fill_rate	DECIMAL(5,2)		92–100% of ordered units actually delivered

supplier_risk_profile

Composite risk assessment per supplier. One row per supplier (not time-series). Updated by the Supplier Evaluation Agent after each assessment cycle.

Column Name	Data Type	Key	Description / Constraints
risk_id	INT	PK	Surrogate primary key
supplier_id	INT	FK	References suppliers(supplier_id)
financial_risk_score	DECIMAL(5,2)		Likelihood of supplier insolvency or payment issues
operational_risk_score	DECIMAL(5,2)		Production capacity and quality consistency risk
geographic_risk_score	DECIMAL(5,2)		Supply chain disruption risk by region
overall_risk_score	DECIMAL(5,2)		Composite score — Policy Approval Agent blocks high-risk POs

6. Entity-Relationship Diagram

The ER diagram below describes all 18 foreign-key relationships across the 4 databases. Relationships are split into intra-database links (within the same DB cluster) and cross-database links (which are the architectural joins that connect the system end-to-end).

6.1 Cross-Database Architecture

Two cross-cluster relationships are the backbone of the entire system:

- `products.sku_id` is referenced by both `inventory_positions` and `in_transit_inventory` in DB3. This means every stock record and every shipment record is traceable back to a specific SKU with its velocity, category, hazmat flags, and seasonality.
- `product_categories.category_id` is referenced by `supplier_category_mapping` and `supplier_pricing_tiers` in DB4. This allows the Procurement Agent to find all eligible suppliers for a given category's reorder and apply the correct volume discount.

6.2 Full Relationship Table

Cardinality notation: 1 = one side (parent), N = many side (child). Every relationship here is a One-to-Many (1:N) unless noted as a derived view.

Parent Table	Card.	→	Child Table	Join Key	Relationship Meaning
states	1	→	distribution_centers	<code>state_id</code> → <code>state_id</code>	One state contains many DCs
states	1	→	stores	<code>state_id</code> → <code>state_id</code>	One state contains many stores
states	1	→	locations	<code>state_id</code> → <code>state_id</code>	State referenced by location master
store_formats	1	→	stores	<code>format_id</code> → <code>format_id</code>	One format assigned to many stores
distribution_centers	1	→	stores	<code>dc_id</code> → <code>dc_id</code>	Each DC services multiple stores
product_categories	1	→	products	<code>category_id</code> → <code>category_id</code>	Each category has many SKUs
velocity_classes	1	→	products	<code>velocity_id</code> → <code>velocity_id</code>	Each velocity class applies to many SKUs
seasonal_patterns	1	→	products	<code>season_id</code> → <code>season_id</code> (NULL ok)	Optional seasonality per SKU
products	1	→	inventory_positions	<code>sku_id</code> → <code>sku_id</code>	Each product tracked at multiple locations

Parent Table	Card.	→	Child Table	Join Key	Relationship Meaning
products	1	→	in_transit_inventory	sku_id → sku_id	Each product may have multiple shipments
inventory_positions	1	→	inventory_health	Derived view	VIEW reads inventory_positions
payment_terms	1	→	suppliers	payment_term_id → payment_term_id	Payment terms assigned to suppliers
suppliers	1	→	supplier_category_mapping	supplier_id → supplier_id	Supplier mapped to categories with lead times
suppliers	1	→	supplier_pricing_tiers	supplier_id → supplier_id	Volume discount tiers per supplier
suppliers	1	→	supplier_performance_metrics	supplier_id → supplier_id	Historical performance evaluations
suppliers	1	→	supplier_risk_profile	supplier_id → supplier_id	Risk scores per supplier
product_categories	1	→	supplier_category_mapping	category_id → category_id	Which suppliers serve each category
product_categories	1	→	supplier_pricing_tiers	category_id → category_id	Pricing tiers per category

6.3 ER Diagram — Cluster Descriptions

DB1 Cluster — Stores & Warehouses

states sits at the root of DB1. All geographic entities — distribution_centers, stores, and locations — reference it. The stores table is the densest node in this cluster, holding three foreign keys (state_id, format_id, dc_id). The locations master table is a deliberate denormalization: instead of requiring downstream tables to union stores and DCs when looking up a location, all 53 physical sites are pre-unified here with a location_type discriminator.

DB2 Cluster — Product Catalog

product_categories, velocity_classes, and seasonal_patterns are all pure lookup tables with no parents. They converge on products, which is the leaf node of DB2 and the most widely referenced table in the system. The products table carries season_id as a nullable FK — this is intentional; year-round SKUs do not have seasonality and should not be forced into a null-season category just to satisfy referential integrity.

DB3 Cluster — Inventory Position

Both inventory_positions and in_transit_inventory reference products.sku_id, making products the cross-cluster bridge between DB2 and DB3. Neither table carries an explicit FK to locations — this is a performance decision at 265,000 rows. The location_id column is effectively a soft reference validated at

application layer. The `inventory_health` view is a computed projection over `inventory_positions`; it does not store any additional data and requires no join.

DB4 Cluster — Supplier Management

`payment_terms` and the cross-DB `product_categories` are the two parent tables feeding DB4. `suppliers` is the hub — all five supplier sub-tables (`category_mapping`, `pricing_tiers`, `performance_metrics`, `risk_profile`) reference `suppliers.supplier_id`. This star topology means the Supplier Evaluation Agent can aggregate across all dimensions of a vendor with a single join to `suppliers`. The cross-cluster link from `product_categories` into `supplier_category_mapping` and `supplier_pricing_tiers` allows Procurement Agent queries to go: `product` → `category` → eligible suppliers → pricing tiers in a single query chain.

7. Agent-to-Database Mapping

Each of the 8 agents has a primary set of tables it reads from and writes to. This section explains not just which tables each agent uses, but why — describing the operational logic that each agent executes against the database.

7.1 Summary Table

Agent	Primary Role	Primary Tables	How It Uses the Data
Inventory Monitoring Agent	Real-time stock surveillance	inventory_positions, inventory_health	Scans all 265k records each cycle. Uses inventory_health VIEW to classify status. Raises events for ZERO_STOCK and BELOW_SAFETY records.
Demand Forecast Agent	Predict future stock needs	products, velocity_classes, seasonal_patterns, inventory_positions	Joins velocity and seasonality to weight baseline demand. Reads current_stock to calculate days-of-cover.
Replenishment Planning Agent	Raise purchase orders	inventory_positions, in_transit_inventory, products	Compares available_stock to reorder_point. Subtracts in-transit qty before raising a new PO. Uses min_order_qty and case_pack_size.
Transfer Optimization Agent	Balance stock across network	inventory_positions, locations, in_transit_inventory	Finds OVERSTOCK locations and matches them to BELOW_SAFETY locations. Checks in_transit before recommending a transfer.
Supplier Evaluation Agent	Score and rank vendors	suppliers, supplier_performance_metrics, supplier_pricing_tiers, supplier_risk_profile	Aggregates latest metrics per supplier. Weighs on-time delivery, quality, fill-rate, and risk scores.
Policy Approval Agent	Gate-check PO proposals	suppliers, payment_terms, supplier_risk_profile	Validates contract dates, checks overall_risk_score thresholds, confirms active_flag before approving.
Procurement Agent	Issue final purchase orders	suppliers, supplier_pricing_tiers, supplier_category_mapping	Selects best supplier per category, applies volume discount tier, constructs PO with correct lead_time_days.
Notification Agent	Broadcast alerts & status	Reads all agent outputs, in_transit_inventory, inventory_health	Monitors transit_status for DELAYED/LOST. Reads VIEW for stock alerts. Dispatches to store managers, DC operators.

7.2 Agent Execution Flow

The agents operate in a coordinated sequence triggered by the Inventory Monitoring Agent's scan results:

Step	Agent	Action
1	Inventory Monitoring Agent	Scans inventory_positions / inventory_health. Emits alert events for any non-OPTIMAL records.
2	Demand Forecast Agent	Receives alert context. Projects future demand using velocity and seasonal multipliers.
3	Replenishment Planning Agent	Receives forecast. Calculates required order quantity. Checks in_transit to avoid double-ordering. Drafts PO request.
4	Transfer Optimization Agent	In parallel: checks if OVERSTOCK locations can supply BELOW_SAFETY locations before raising an external PO.
5	Policy Approval Agent	Reviews PO draft. Validates supplier contract status, risk scores, and payment terms. Approves or blocks.
6	Procurement Agent	On approval: selects supplier per category, applies pricing tier, issues final PO with correct lead time.
7	Notification Agent	Broadcasts status updates to relevant stakeholders at each step. Also monitors DELAYED/LOST transit shipments independently.

8. Design Notes & Recommendations

8.1 Schema Validation

Area	Status	Notes
Primary keys	All correct	All 18 tables have defined PKs; BIGINT used appropriately for high-volume tables (inventory_positions, in_transit_inventory, supplier_performance_metrics)
Unique constraints	All correct	sku_code, store_code, supplier_code all carry UQ constraints preventing duplicate business keys
Nullable FKs	Correct by design	products.season_id is intentionally NULL for non-seasonal items — no change needed
Soft location FKs	Acceptable	inventory_positions and in_transit_inventory use soft location_id references for performance at 265k rows — document this in application layer validation
inventory_health VIEW	Good pattern	Centralises classification logic. Recommendation: add max_stock_level to the view output so OVERSTOCK logic is also derived, not duplicated
BIGINT PKs	Correct	Appropriate for tables that accumulate historical rows over time

8.2 Recommended Indexes

Table	Index Column(s)	Reason
inventory_positions	(sku_id, location_id)	Composite index for the primary agent query pattern
inventory_positions	inventory_status	Allows fast WHERE status = 'BELOW_SAFETY' scans
inventory_positions	location_id	Transfer Agent frequently filters by location
in_transit_inventory	(sku_id, destination_location_id)	Replenishment Agent join pattern
in_transit_inventory	transit_status	Notification Agent filters for DELAYED and LOST
supplier_performance_metrics	(supplier_id, evaluation_date)	Evaluation Agent time-series queries
products	category_id	Procurement Agent filters SKUs by category
products	velocity_id	Demand Forecast Agent groups by velocity class

8.3 Product Category Reference

#	Category	Key Attributes
---	----------	----------------

1	Electronics	High retail price (up to \$1,299), low shelf-life risk, A/B velocity, hazmat possible for batteries
2	Apparel	Wide SKU range, seasonal peaks (Back-to-School, Holiday), mostly B/C velocity
3	Home Goods	Medium price, stable demand, mostly B velocity, often large/heavy (weight_lbs matters)
4	Sporting Goods	Strong Summer seasonality, A/B velocity, includes hazmat (aerosols, chemicals)
5	Toys	Strong Holiday Peak seasonality, A velocity for top titles, C for long-tail
6	Health & Beauty	Short shelf_life_days for some items, temperature-controlled for some, moderate velocity
7	Grocery	Shortest shelf_life_days, temperature-controlled, high A velocity, frequent replenishment
8	Seasonal	Defined by season_id — demand_multiplier drives extreme reorder swings

9. Complete Table Inventory

Quick reference listing all 18 tables and 1 view across the 4 databases.

#	Table / View	Database	Purpose
1	states	DB1 — Stores & Warehouses	US state lookup
2	store_formats	DB1 — Stores & Warehouses	Small/Medium/Large format definitions
3	distribution_centers	DB1 — Stores & Warehouses	3 regional DC records
4	stores	DB1 — Stores & Warehouses	50 retail store records
5	locations	DB1 — Stores & Warehouses	Master location lookup (53 records)
6	product_categories	DB2 — Product Catalog	8 category records
7	velocity_classes	DB2 — Product Catalog	A/B/C velocity lookup
8	seasonal_patterns	DB2 — Product Catalog	3 seasonal window definitions
9	products	DB2 — Product Catalog	5,000 SKU master records
10	inventory_positions	DB3 — Inventory Position	265,000 SKU-location stock records
11	in_transit_inventory	DB3 — Inventory Position	Active shipment tracking records
12	inventory_health (VIEW)	DB3 — Inventory Position	Computed status view over inventory_positions
13	payment_terms	DB4 — Supplier Management	NET30/NET60/2-10 NET30 definitions
14	suppliers	DB4 — Supplier Management	35 vendor master records
15	supplier_category_mapping	DB4 — Supplier Management	Supplier-to-category bridge + lead times
16	supplier_pricing_tiers	DB4 — Supplier Management	Volume discount tiers per supplier/category
17	supplier_performance_metrics	DB4 — Supplier Management	Historical performance snapshots
18	supplier_risk_profile	DB4 — Supplier Management	Financial/operational/geographic risk scores

Database 5 - Inventory Events & Event Snapshots

Table 1: inventory_events

Column	Datatype	Description
event_id	BIGINT	Unique Event Identifier
event_type	VARCHAR(50)	Type of inventory movement
sku_id	INT	Product SKU
location_id	INT	Store/DC Location
quantity_change	INT	Positive or negative inventory movement
event_timestamp	TIMESTAMP	Time of event
reference_id	VARCHAR(100)	Source transaction reference
source_location_id	INT	Transfer source location
destination_location_id	INT	Transfer destination location
event_reason	VARCHAR(255)	Business reason
created_by	VARCHAR(100)	Source system

Table 2: inventory_daily_snapshots

Column	Datatype	Description
snapshot_id	BIGINT	Unique Snapshot Identifier
snapshot_date	DATE	Snapshot Date
sku_id	INT	Product SKU
location_id	INT	Store/DC Location
opening_stock	INT	Beginning inventory
receipts	INT	Quantity received
sales	INT	Quantity sold
transfers_in	INT	Inbound transfers
transfers_out	INT	Outbound transfers
adjustments	INT	Inventory corrections
closing_stock	INT	End-of-day inventory